CSD 330 – Security Analytics

HW #3

Kabir Kanha Arora

Practical Exercises

1. **Run SVM classifier with 10-fold cross validation with at least three different kernels (no more than two can be polynomial kernels)**

Answer:

i. Linear Kernel (Polynomial with exponent = 1)

```
Time taken to build model: 15.45 seconds

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances       10370               93.8037 %
Incorrectly Classified Instances       685                6.1963 %
Kappa statistic                          0.8742
Mean absolute error                      0.062
Root mean squared error                  0.2489
Relative absolute error                 12.5554 %
Root relative squared error             50.1107 %
Total Number of Instances            11055

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.920    0.047    0.939      0.920    0.929      0.874    0.936     0.899     -1
                 0.953    0.080    0.937      0.953    0.945      0.874    0.936     0.919     1
Weighted Avg.    0.938    0.066    0.938      0.938    0.938      0.874    0.936     0.910

=== Confusion Matrix ===

    a    b   <-- classified as
 4504  394 |    a = -1
  291 5866 |    b = 1
```

ii. RBFKernel

```
Time taken to build model: 27.39 seconds

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances       10395               94.0299 %
Incorrectly Classified Instances       660                5.9701 %
Kappa statistic                          0.8787
Mean absolute error                      0.0597
Root mean squared error                  0.2443
Relative absolute error                 12.0972 %
Root relative squared error             49.1878 %
Total Number of Instances            11055

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.918    0.042    0.946      0.918    0.932      0.879    0.938     0.905     -1
                 0.958    0.082    0.936      0.958    0.947      0.879    0.938     0.920     1
Weighted Avg.    0.940    0.064    0.940      0.940    0.940      0.879    0.938     0.913

=== Confusion Matrix ===

    a    b   <-- classified as
 4494  404 |    a = -1
  256 5901 |    b = 1
```

iii. Puk

```
Time taken to build model: 112.62 seconds

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances       10737               97.1235 %
Incorrectly Classified Instances       318                2.8765 %
Kappa statistic                          0.9416
Mean absolute error                      0.0288
Root mean squared error                  0.1696
Relative absolute error                  5.8286 %
Root relative squared error             34.1428 %
Total Number of Instances            11055

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.959    0.019    0.976      0.959    0.967      0.942    0.970     0.954     -1
                 0.981    0.041    0.968      0.981    0.974      0.942    0.970     0.960     1
Weighted Avg.    0.971    0.031    0.971      0.971    0.971      0.942    0.970     0.957

=== Confusion Matrix ===

    a    b   <-- classified as
 4697  201 |    a = -1
  117 6040 |    b = 1
```

- ➔ **So we see that the Puk kernel performs the best, it is also interesting to note that building this model took far more time that the other two.**
- ➔ **For more details, refer to the attached text files.**

2. On the same dataset, can you use the MLP classifier. If yes, try a 3-layer model. If not, explain why you can't use it.

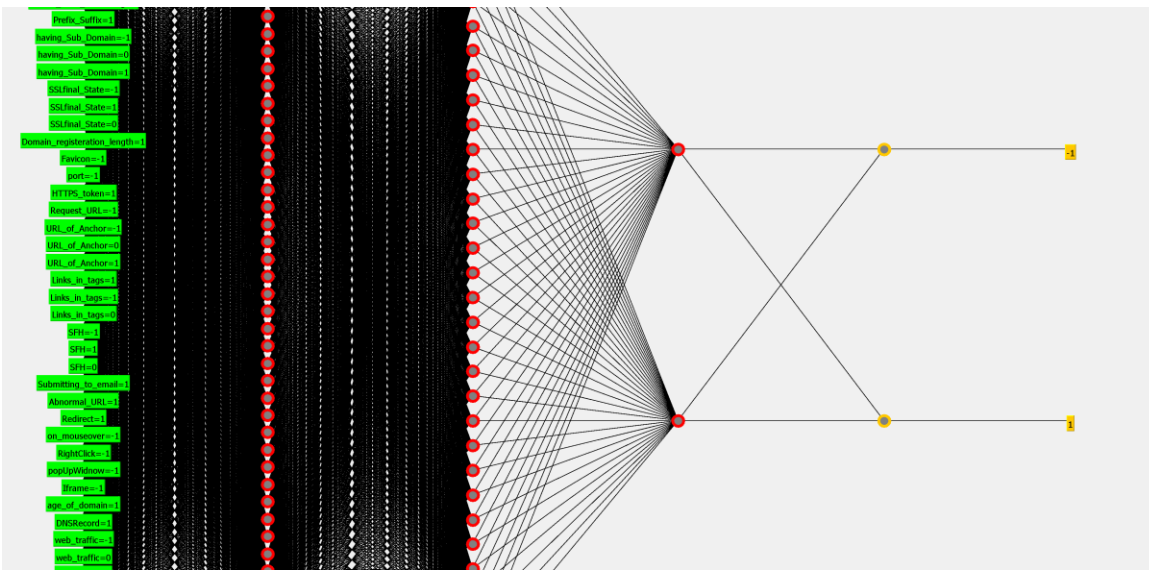
Answer:

Yes, Weka has the option of using a MultilayerPerceptron for classification.

I chose 46 neurons for the input layer since there are 46 features, 2 neurons for the output layer, since there are two classes we need to classify into, and chose the number of hidden layer neurons as 24 according to the formula *Mean of input layer and output layer*. I referred to the given link for help.

https://stats.stackexchange.com/questions/181/how-to-choose-the-number-of-hidden-layers-and-nodes-in-a-feedforward-neural-netw

Refer to txt file for more details.



(The entire visualisation could not fit in one screenshot)

```
Time taken to build model: 187.47 seconds

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances       10703               96.8159 %
Incorrectly Classified Instances       352                3.1841 %
Kappa statistic                          0.9354
Mean absolute error                      0.0352
Root mean squared error                  0.158
Relative absolute error                  7.1244 %
Root relative squared error             31.809  %
Total Number of Instances            11055

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.958    0.023    0.970      0.958    0.964      0.935    0.989     0.987     -1
                 0.977    0.042    0.967      0.977    0.972      0.935    0.989     0.988     1
Weighted Avg.    0.968    0.034    0.968      0.968    0.968      0.935    0.989     0.988

=== Confusion Matrix ===

    a    b   <-- classified as
 4690  208 |    a = -1
  144 6013 |    b = 1
```

3. **Download and install LIBOL for online learning and try different online learning algorithms including: AROW, Passive Aggressive and Confidence Weighted algorithm (if available). It will be easier to install the software if you install a Git client first. Try controlling the number of iterations to 10, 100, 1000 (compare the results). http://libol.stevenhoi.org/LIBOL manual.pdf**

Answer: *Refer to attached text files*

4. **Try Adaboost with SVM as the base classifier and compare the results with those in item 1 above.**

Answer:

```
Time taken to build model: 249.07 seconds

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances       10370               93.8037 %
Incorrectly Classified Instances       685                6.1963 %
Kappa statistic                          0.8742
Mean absolute error                      0.0808
Root mean squared error                  0.23
Relative absolute error                 16.3691 %
Root relative squared error             46.2961 %
Total Number of Instances            11055

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.920    0.047    0.939      0.920    0.929      0.874    0.982     0.979     -1
                 0.953    0.080    0.937      0.953    0.945      0.874    0.982     0.984     1
Weighted Avg.    0.938    0.066    0.938      0.938    0.938      0.874    0.982     0.982

=== Confusion Matrix ===

    a    b   <-- classified as
 4504  394 |    a = -1
  291 5866 |    b = 1
```

The first comparison right off the bat is that AdaBoost takes ages to build the model. While the other models in Q1 took negligible time, AdaBoost took a good 20 minutes to do the same. It gives results very similar to SVM with linear kernel, and worse than that with RBF and Puk. Refer to txt file for more details.